

Geometriemodelle in der Bauinformatik SoSe25

Triangulation, Visualisierung und topologische  
Analyse von 3D-Polyedern

Immanuel Michalak  
Matrikelnummer: 485910

# Inhaltsverzeichnis

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Theoretical Foundations</b>                                     | <b>1</b>  |
| 1.1      | Boundary Representation (B-Rep) und 2D-Projektion . . . . .        | 1         |
| 1.1.1    | Grundlagen der Boundary Representation . . . . .                   | 1         |
| 1.1.2    | Topologische Elemente und Orientierung . . . . .                   | 1         |
| 1.1.3    | Projektion von 3D-Flächen in 2D-Koordinatensysteme . . . . .       | 1         |
| 1.2      | Delaunay-Triangulation und Rücktransformation . . . . .            | 2         |
| 1.2.1    | Prinzip der Delaunay-Triangulation . . . . .                       | 2         |
| 1.2.2    | Triangulation mit Poly2Tri . . . . .                               | 2         |
| 1.2.3    | Mapping der Dreiecke zurück in 3D . . . . .                        | 2         |
| 1.3      | Breitensuche (BFS) zur Nachbarschaftsanalyse . . . . .             | 3         |
| 1.3.1    | Grundidee und Ablauf der BFS . . . . .                             | 3         |
| 1.3.2    | Topologische Verknüpfung durch Twin-Kanten . . . . .               | 3         |
| 1.3.3    | Farbkodierte Visualisierung nach topologischer Distanz . . . . .   | 3         |
| <b>2</b> | <b>System Design</b>                                               | <b>5</b>  |
| 2.1      | Klassendiagramm der Systemarchitektur . . . . .                    | 5         |
| 2.2      | Datenfluss und View-Model-Struktur . . . . .                       | 7         |
| 2.2.1    | Datenfluss vom Modell zur Visualisierung . . . . .                 | 7         |
| 2.2.2    | View-Model-Verknüpfung . . . . .                                   | 8         |
| 2.2.3    | Unterschiede zwischen JavaFX und jMonkeyEngine . . . . .           | 8         |
| <b>3</b> | <b>Algorithmen</b>                                                 | <b>10</b> |
| 3.1      | Pseudocode: Nachbarschaftsanalyse per Breitensuche (BFS) . . . . . | 10        |
| 3.2      | Mesh-Generierung und Mapping-Pipeline . . . . .                    | 11        |
| 3.3      | Topologischer Test mittels Euler-Poincaré-Formel . . . . .         | 11        |
| <b>4</b> | <b>Unit Tests</b>                                                  | <b>13</b> |
| 4.1      | Teststrategie . . . . .                                            | 13        |
| 4.2      | Testfälle . . . . .                                                | 13        |

# Abbildungsverzeichnis

|   |                                                                                                                |   |
|---|----------------------------------------------------------------------------------------------------------------|---|
| 1 | Farbliche Darstellung der Flächen eines Polyeder basierend auf der Nachbarschaftsanalyse mittels BFS . . . . . | 4 |
| 2 | UML-Diagramm des Pakets <code>model</code> . . . . .                                                           | 5 |
| 3 | UML-Diagramm des Pakets <code>logic</code> . . . . .                                                           | 5 |
| 4 | UML-Diagramm des Pakets <code>projection</code> . . . . .                                                      | 6 |
| 5 | UML-Diagramm des Pakets <code>app</code> . . . . .                                                             | 6 |
| 6 | UML-Diagramm des Pakets <code>jme3D</code> . . . . .                                                           | 6 |
| 7 | UML-Diagramm des Pakets <code>fx3D</code> . . . . .                                                            | 7 |

# 1 Theoretical Foundations

## 1.1 Boundary Representation (B-Rep) und 2D-Projektion

### 1.1.1 Grundlagen der Boundary Representation

Die Boundary Representation (B-Rep) ist ein etabliertes Modell zur Beschreibung von Volumenkörpern über deren Begrenzungen. Dabei wird ein Körper nicht durch sein Volumen, sondern durch die ihn umschließenden Flächen definiert. Diese Flächen setzen sich aus Rändern (Edges) zusammen, welche wiederum durch Punkte (Vertices) verbunden sind.

Im Rahmen dieser Hausarbeit kommt eine *Half-Edge-Datenstruktur* zum Einsatz, um die topologischen Beziehungen zwischen den geometrischen Elementen effizient abzubilden. Jeder Halfkante (HalfEdge) ist ein Ursprungspunkt (origin), eine nächste Kante (next), eine vorherige Kante (prev), ein Flächenobjekt (face) zu dem die Kante gehört und eine Zwillingskante (twin) zugeordnet. Diese Struktur erlaubt eine konsistente Navigation innerhalb eines Polyeders, z. B. zur Traversierung einer Fläche oder zur Nachbarschaftsanalyse zwischen benachbarten Flächen.

### 1.1.2 Topologische Elemente und Orientierung

Ein Polyeder besteht aus einer Menge von *Faces*, *HalfEdges* und *Points*. Die Orientierung der HalfEdges innerhalb einer Fläche erfolgt gegen den Uhrzeigersinn, sodass die Normalenvektoren der Flächen nach außen gerichtet sind. Diese Konvention ist entscheidend für die Konsistenz der Normalenrichtung und somit für die korrekte Visualisierung und Topologieprüfung.

Die Schleife aus HalfEdges bildet den äußeren Rand einer Fläche, während optionale innere Schleifen zur Darstellung von Löchern verwendet werden können. Die Verknüpfung über `twin`-Referenzen erlaubt das Navigieren zwischen benachbarten Flächen, was für die spätere Nachbarschaftsanalyse per BFS essentiell ist.

### 1.1.3 Projektion von 3D-Flächen in 2D-Koordinatensysteme

Um eine Fläche triangulieren zu können, muss sie zunächst aus dem dreidimensionalen Raum in eine zweidimensionale Ebene überführt werden. Dies ist notwendig, da die verwendete Delaunay-Triangulation in einem 2D-Raum operiert. Für jede Fläche wird ein lokales zweidimensionales Koordinatensystem konstruiert, dessen Ebene parallel zur Fläche liegt. Die 3D-Punkte der Fläche werden orthogonal in dieses lokale System projiziert, um eine planare Darstellung für die Triangulation zu ermöglichen.

## 1.2 Delaunay-Triangulation und Rücktransformation

### 1.2.1 Prinzip der Delaunay-Triangulation

Die Delaunay-Triangulation ist ein Verfahren zur Zerlegung einer Punktmenge in ein Netz aus Dreiecken, das bestimmte geometrische Eigenschaften optimiert. Ein wesentliches Kriterium dabei ist, dass der Umkreis eines jeden Dreiecks keine weiteren Punkte der Triangulation enthält. Dies führt typischerweise zu gut geformten Dreiecken mit möglichst großen Mindestwinkeln, was numerische Stabilität und eine gleichmäßige Vernetzung begünstigt.

Im Kontext dieser Hausarbeit dient die Delaunay-Triangulation zur Aufteilung projektierter, planarer Polygonflächen – inklusive eventueller Löcher – in einzelne Dreiecke, um sie für die Visualisierung und Weiterverarbeitung nutzbar zu machen.

### 1.2.2 Triangulation mit Poly2Tri

Zur Umsetzung wird die Open-Source-Bibliothek `Poly2Tri` verwendet. Diese erlaubt die Triangulation von planaren Polygonen unter Berücksichtigung von Löchern (inner loops). In der konkreten Implementierung erfolgt die Triangulation in mehreren Schritten:

1. **Grenzen erfassen:** Zunächst werden die äußeren und inneren Ränder der Fläche (Face) gesammelt. Dabei wird die äußere Grenze über `face.getOrderedBoundaryPoints()` und etwaige Löcher über `Face.getPointsFromStartHalfEdge()` erfasst.
2. **2D-Projektion:** Die gesammelten 3D-Konturen werden mit Hilfe von `PolygonProjection.projectTo2D` in ein lokales, flächennahes 2D-Koordinatensystem projiziert. Dabei entsteht eine Liste von `PolygonPoint`-Objekten sowie eine Zuordnung zu den ursprünglichen 3D-Punkten über eine Map.
3. **Triangulation:** Das äußere Polygon und seine Löcher werden an `Poly2Tri` übergeben. Die Bibliothek berechnet daraus eine konforme Dreiecksvernetzung.

Die Trennung von Geometrie (3D) und Triangulationslogik (2D) erlaubt eine robuste Behandlung beliebig komplexer Flächen.

### 1.2.3 Mapping der Dreiecke zurück in 3D

Nach erfolgreicher Triangulation liegen die resultierenden Dreiecke im lokalen 2D-Koordinatensystem vor. Um diese jedoch in der ursprünglichen 3D-Visualisierung darstellen zu können, müssen sie in den 3D-Raum zurücktransformiert werden.

Dieser Schritt erfolgt über eine einfache Zuordnung: Während der Projektion wurde ein Mapping von jedem 2D-Punkt auf seinen ursprünglichen 3D-Punkt gespeichert. Mit Hilfe dieses Mappings

kann jeder Eckpunkt eines Delaunay-Dreiecks in 2D eindeutig auf den korrespondierenden Punkt im 3D-Raum zurückgeführt werden.

Dadurch entsteht eine Liste von 3D-Dreiecken, welche die jeweilige Fläche des Polyeders trianguliert wiedergeben. Diese Dreiecke können anschließend direkt in die Mesh-Modelle der JavaFX- bzw. jMonkey-Visualisierung überführt werden.

### 1.3 Breitensuche (BFS) zur Nachbarschaftsanalyse

#### 1.3.1 Grundidee und Ablauf der BFS

Die Breitensuche (Breadth-First Search, BFS) ist ein klassischer Algorithmus zur Traversierung von Graphen. Im Gegensatz zur Tiefensuche erkundet BFS die Nachbarn eines Knotens vollständig, bevor es in tiefere Ebenen vorstößt. Dadurch eignet sich der Algorithmus besonders gut zur Berechnung topologischer Distanzen, z. B. zur Analyse von Nachbarschaften innerhalb eines Polyeders.

In dieser Anwendung wird BFS verwendet, um von einer selektierten Fläche (**Face**) ausgehend alle angrenzenden Flächen zu bestimmen. Die Flächen werden dabei nach ihrer Entfernung in „Leveln“ organisiert: direkte Nachbarn sind Level 1, deren Nachbarn Level 2 usw. Die Implementierung basiert auf einer Queue-Datenstruktur, in der noch zu besuchende Flächen gespeichert werden. Bereits besuchte Flächen werden über eine Map markiert, um doppelte Besuche zu vermeiden.

#### 1.3.2 Topologische Verknüpfung durch Twin-Kanten

Die Grundlage für die Nachbarschaftsanalyse bildet die **HalfEdge**-Struktur, insbesondere deren **twin**-Beziehung. Jede Kante besitzt einen Zwillingszeiger auf die angrenzende Fläche, sodass von einer Fläche aus über ihre äußeren Kanten gezielt auf benachbarte Flächen zugegriffen werden kann.

Dieser Mechanismus erlaubt eine effiziente Traversierung der Oberfläche des Polyeders. Der BFS-Algorithmus iteriert dabei über alle **HalfEdges** einer Fläche, prüft für jede auf den Zwillingszeiger, und fügt die angrenzende Fläche gegebenenfalls zur Queue hinzu.

#### 1.3.3 Farbkodierte Visualisierung nach topologischer Distanz

Das Ergebnis der BFS-Analyse ist eine Zuordnung jeder besuchten Fläche zu einem bestimmten Level. Diese Information wird genutzt, um die Flächen farblich zu unterscheiden – typischerweise durch eine Farbskala, bei der jede Ebene eine eigene Farbe erhält. Dadurch wird die topologische Distanz visuell erfassbar.

Ein typisches Beispiel ist der Würfel: Wird eine beliebige Fläche ausgewählt (Level 0), so besitzen die vier direkt angrenzenden Seitenflächen Level 1, die gegenüberliegende Fläche Level 2. Diese

Information kann sowohl in JavaFX als auch in jMonkeyEngine visuell dargestellt werden, etwa durch Zuweisung unterschiedlicher Materialfarben zu den gerenderten Meshes oder Geometrien.

Die farbliche Kodierung der Flächen basiert auf einer Normalisierung der BFS-Level relativ zum höchsten gefundenen Level. Für jedes Level  $l$  wird ein normierter Wert  $t = \frac{l}{l_{\max}}$  berechnet. Dieser Wert dient als Parameter für einen Farbverlauf im HSB-Farbraum – von Blau (Level 0) bis Rot (höchstes Level). Die Farbzweisung erfolgt über eine Funktion `hueGradient(t)`, die den Farbton linear im Bereich von  $240^\circ$  (Blau) bis  $0^\circ$  (Rot) interpoliert. Die berechnete Farbe wird anschließend in das jeweilige Materialsystem der Visualisierungsumgebung überführt.

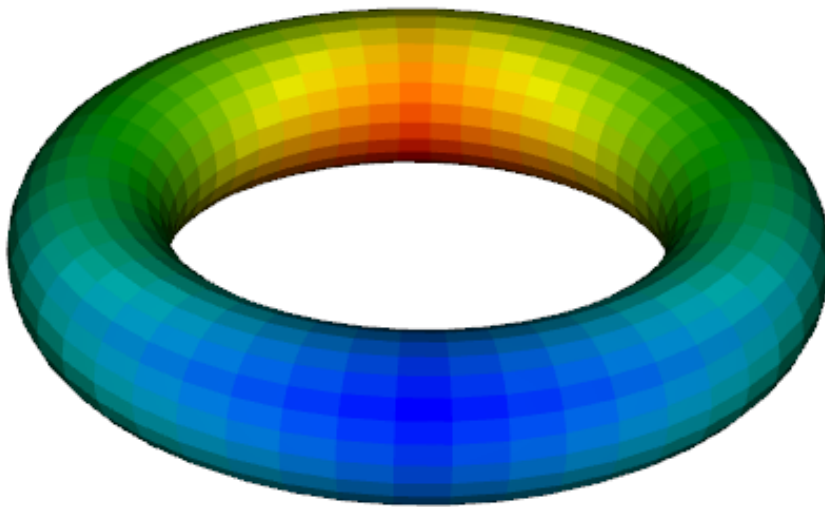


Abbildung 1: Farbliche Darstellung der Flächen eines Polyeder basierend auf der Nachbarschaftsanalyse mittels BFS

## 2 System Design

### 2.1 Klassendiagramm der Systemarchitektur

Zur Veranschaulichung der Struktur des entwickelten Softwaresystems wurden UML-Diagramme erstellt. Sie zeigen die zentralen Klassen der einzelnen Pakete sowie deren Beziehungen untereinander. Die Diagramme verdeutlichen, wie Datenmodelle, logische Operationen und Visualisierungskomponenten in modularer Weise miteinander verknüpft sind.

Um die Lesbarkeit zu gewährleisten, wurde für jedes Java-Package ein eigenes Klassendiagramm erzeugt.

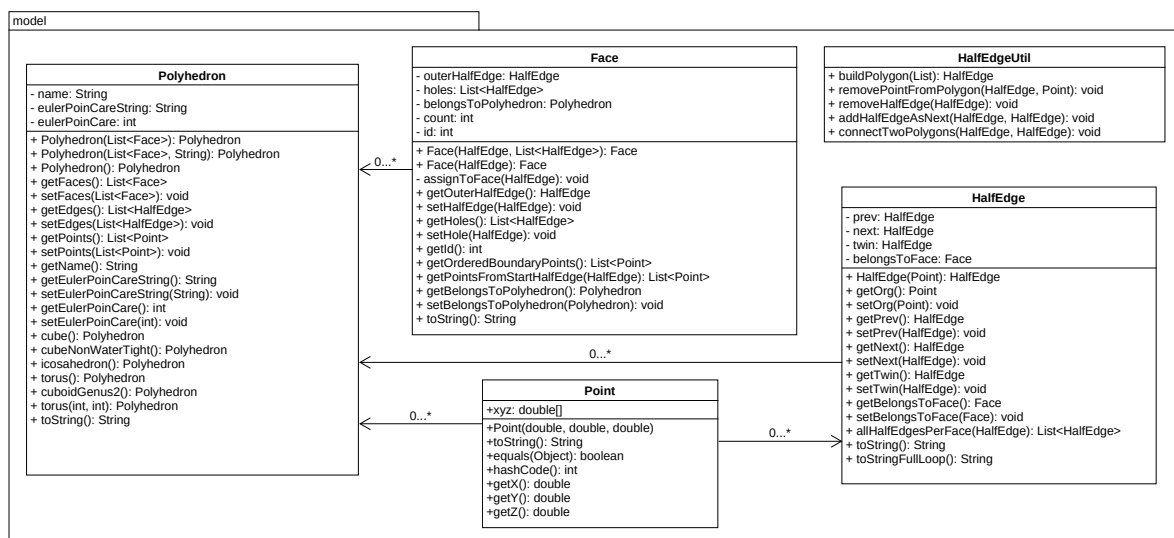


Abbildung 2: UML-Diagramm des Packets model

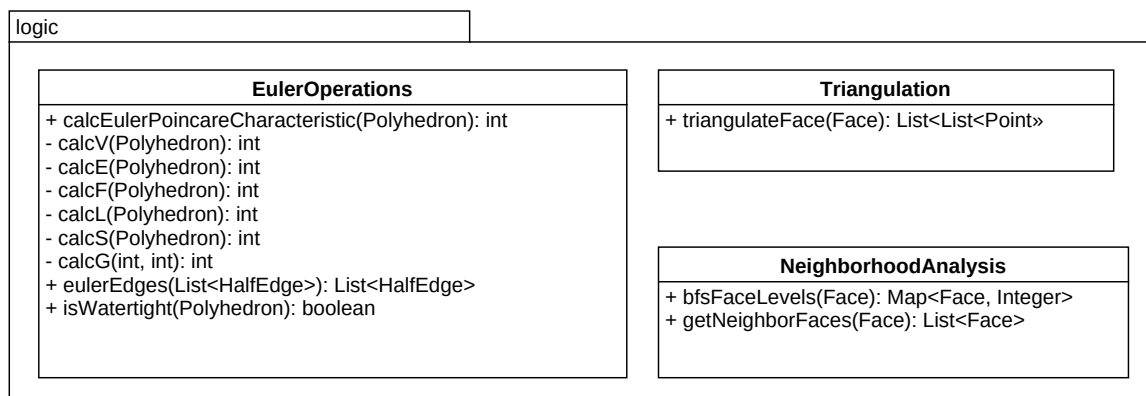
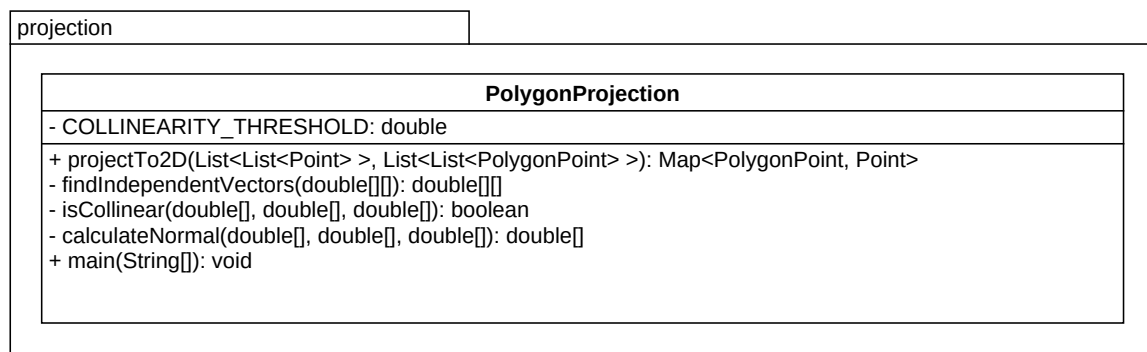
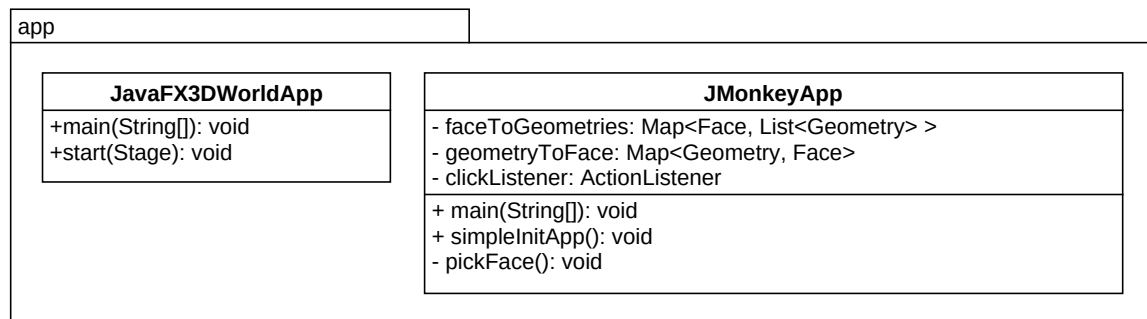
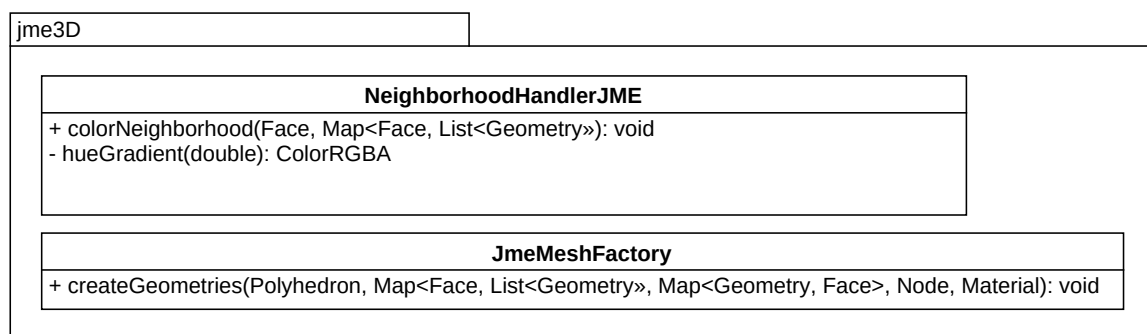
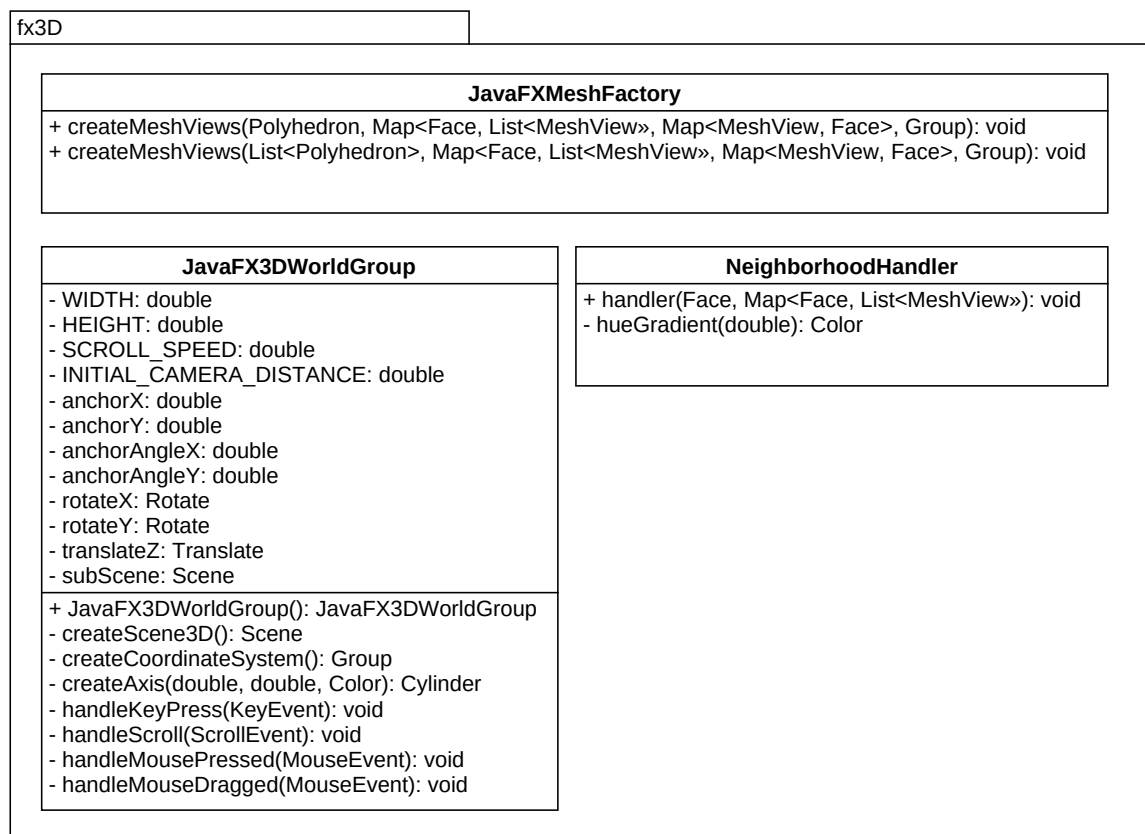


Abbildung 3: UML-Diagramm des Packets logic

Abbildung 4: UML-Diagramm des Packets **projection**Abbildung 5: UML-Diagramm des Packets **app**Abbildung 6: UML-Diagramm des Packets **jme3D**



Abbildung 7: UML-Diagramm des Packets `fx3D`

## 2.2 Datenfluss und View-Model-Struktur

### 2.2.1 Datenfluss vom Modell zur Visualisierung

Der Datenfluss im System folgt einem modular aufgebauten Ablauf, der die strikte Trennung von Datenmodell, Logik und Darstellung berücksichtigt. Ausgangspunkt ist ein oder mehrere `Polyhedron`-Objekte, die als strukturierte Datenträger auf Basis der Half-Edge-Repräsentation definiert sind. Diese enthalten alle topologischen Informationen über Flächen, Kanten und Punkte.

Im ersten Schritt erfolgt die Umwandlung dieser Modellstruktur in eine darstellbare Geometrie. Dazu wird für jede `Face` eines Polyeders die Methode `Triangulation.triangulateFace(Face)` aufgerufen. Die Methode extrahiert die Randpunkte der Fläche, projiziert sie in ein lokales 2D-Koordinatensystem und trianguliert das Polygon mithilfe der Bibliothek `Poly2Tri`. Anschließend werden die resultierenden 2D-Dreiecke wieder in den 3D-Raum zurückübertragen.

Die Liste von 3D-Dreiecken wird dann in einem der beiden Visualisierungs-Frameworks verarbeitet. In JavaFX erfolgt dies durch `JavaFXMeshFactory`, in `jMonkeyEngine` durch `JmeMeshFactory`. Beide Klassen erstellen für jede Fläche jeweils eine Liste von Meshes bzw. Geometrien, die aus

den Dreieckspunkten aufgebaut werden. Zusätzlich werden sie mit einem Material versehen und in die jeweilige Szenengraph-Struktur eingefügt.

Parallel zur geometrischen Darstellung wird die Verbindung zwischen Datenmodell und Darstellung aufrechterhalten: Über Mapping-Strukturen wie `Map<Face, List<MeshView>>` (JavaFX) bzw. `Map<Face, List<Geometry>>` (jME) können spätere Interaktionen wie z. B. Flächenselektion oder Färbung eindeutig auf die entsprechenden Modellobjekte zurückgeführt werden.

Der Datenfluss ist somit klar strukturiert: Modell → Logik → Darstellung, wobei jeder Schritt bewusst gekapselt und unabhängig voneinander gehalten ist.

### 2.2.2 View-Model-Verknüpfung

Für eine interaktive Visualisierung reicht es nicht aus, die triangulierten Flächen lediglich grafisch darzustellen. Es muss zusätzlich eine Verbindung zwischen der zugrunde liegenden Modellstruktur und den erzeugten View-Elementen bestehen, um Interaktionen – etwa Selektion, Nachbarschaftsanalyse oder farbliche Markierung – auf das ursprüngliche Datenmodell zurückführen zu können.

Diese Verknüpfung erfolgt über bidirektionale Mapping-Strukturen: Für jede Fläche (`Face`) wird eine Liste der zugehörigen grafischen Elemente gespeichert. In JavaFX sind dies `MeshView`-Objekte, in jMonkeyEngine `Geometry`-Instanzen. Die Zuordnung erfolgt über die Maps `Map<Face, List<MeshView>>` bzw. `Map<Face, List<Geometry>>`. Zusätzlich wird die inverse Beziehung gespeichert, also `Map<MeshView, Face>` bzw. `Map<Geometry, Face>`, um bei Benutzerinteraktionen schnell vom View-Element auf das zugrundeliegende Modellobjekt zugreifen zu können.

Dieses Mapping wird während der Geometrieerzeugung im Rahmen der Methoden `createMeshViews` (JavaFX) bzw. `createGeometries` (jMonkeyEngine) automatisch aufgebaut. Jeder Triangulationsschritt erzeugt ein oder mehrere grafische Elemente, die umgehend mit dem zugehörigen `Face` verknüpft werden. Diese strukturierte Kopplung erlaubt nicht nur eine spätere Analyse, sondern stellt auch die Grundlage für dynamische Farbänderungen während der Interaktion dar.

Durch diese saubere Trennung und explizite Verknüpfung von Model- und View-Schicht bleibt die Architektur erweiterbar und wartbar. Logische Operationen wie die BFS-Analyse oder der Euler-Test können direkt auf das Datenmodell zugreifen, während visuelle Rückmeldungen synchron auf View-Ebene erfolgen.

### 2.2.3 Unterschiede zwischen JavaFX und jMonkeyEngine

Obwohl beide Visualisierungsframeworks auf die gleiche Datenbasis und die identische Logikschicht zugreifen, unterscheiden sie sich in ihrer konkreten Umsetzung der Darstellung.

In JavaFX basiert die Darstellung auf `MeshView`-Objekten, die in eine `SubScene` eingefügt und über eine hierarchische `Group`-Struktur organisiert werden. Die Kamera- und Benutzerinteraktion wird über eine eigens entwickelte Klasse (`JavaFX3DWorldGroup`) realisiert, die Mausinter-

aktionen und Tasteneingaben in Transformationen des Viewports übersetzt.

jMonkeyEngine verwendet dagegen einen leistungsfähigeren Scenegraph mit **Geometry**-Objekten und direkter Low-Level-Zugriffsmöglichkeit auf Vertex- und Index-Puffer über die Klassen **Mesh** und **BufferUtils**. Die Kamerasteuerung sowie das Ray-basierte Picking für die Auswahl von Flächen ist hier integraler Bestandteil der Engine und ermöglicht eine hochperformante Interaktion mit komplexeren Szenen.

Trotz dieser technischen Unterschiede bleibt die zugrundeliegende Architektur identisch: Die gleiche Polyhedron-Struktur wird trianguliert, in grafische Primitive überführt und über ein Mapping mit dem Modell verknüpft. Damit zeigt sich die Stärke der gewählten Trennung zwischen Modell-, Logik- und Darstellungsebene: Änderungen in der Visualisierungsschicht erfordern keine Modifikation am Datenmodell oder den Analysealgorithmen.

### 3 Algorithmen

#### 3.1 Pseudocode: Nachbarschaftsanalyse per Breitensuche (BFS)

Zur Bestimmung der topologischen Entfernung von Flächen wird ein modifizierter Breitensuchalgorithmus (BFS) auf der Struktur des Polyeders durchgeführt. Dabei wird von einer Startfläche ausgehend über die `twin`-Beziehungen zu benachbarten Flächen navigiert. Jede neu entdeckte Fläche erhält ein Level, das ihrer Entfernung zur Startfläche entspricht.

```
BFS-Face-Level(startFace):  
    faceLevel ← leeres Map<Face, Integer>  
    queue ← leere Queue<Face>  
  
    faceLevel[startFace] ← 0  
    queue.enqueue(startFace)  
  
    while queue ist nicht leer:  
        current ← queue.dequeue()  
        currentLevel ← faceLevel[current]  
  
        for each neighbor in getNeighborFaces(current):  
            if neighbor nicht in faceLevel:  
                faceLevel[neighbor] ← currentLevel + 1  
                queue.enqueue(neighbor)  
  
    return faceLevel  
  
getNeighborFaces(face):  
    neighbors ← leere Set<Face>  
  
    for each halfEdge in outer und hole-boundaries von face:  
        twin ← halfEdge.twin  
        if twin existiert und twin.face != face:  
            neighbors.add(twin.face)  
  
    return neighbors
```

Der Algorithmus stellt sicher, dass jede Fläche genau einmal besucht wird, und erlaubt somit eine effiziente Topologie-Analyse zur farblichen Visualisierung der Nachbarschaftstiefe.

### 3.2 Mesh-Generierung und Mapping-Pipeline

Nach der Triangulation der Flächen in 3D-Dreiecke erfolgt deren Umwandlung in darstellbare Geometrieobjekte für die jeweilige Visualisierungsumgebung. Dieser Schritt wird sowohl für JavaFX als auch für jMonkeyEngine in getrennten Fabrikklassen ausgeführt (`JavaFXMeshFactory` bzw. `JmeMeshFactory`).

**JavaFX:** Für jede Fläche (`Face`) wird zunächst eine Liste an 3D-Dreiecken erzeugt. Jedes dieser Dreiecke besteht aus drei `Point`-Objekten. Daraus wird ein `TriangleMesh` aufgebaut, welches in ein `MeshView`-Objekt überführt wird. Jedes `MeshView` erhält zusätzlich ein `PhongMaterial`, das später für die Interaktion (z. B. Färbung) angepasst werden kann. Die erzeugten Meshes werden in den Szenengraph (`Group`) eingefügt.

**jMonkeyEngine:** In jME erfolgt die Geometrieerzeugung durch direkten Zugriff auf die Vertex- und Indexpuffer. Aus den Punkten jedes Dreiecks wird ein `Mesh`-Objekt konstruiert, welches anschließend in eine `Geometry` eingebettet wird. Diese erhält eine Kopie des Standard-Materials, sodass individuelle Farbanpassungen möglich sind. Die `Geometry`-Objekte werden dem `Node` des Szenengraphen hinzugefügt.

**Mapping zwischen Modell und Darstellung:** Unabhängig vom Framework wird für jede Fläche eine eindeutige Zuordnung zu den zugehörigen Mesh-Objekten aufgebaut. Diese erfolgt über zwei bidirektionale Datenstrukturen:

- `Map<Face, List<MeshView>>` bzw. `Map<Face, List<Geometry>>`
- `Map<MeshView, Face>` bzw. `Map<Geometry, Face>`

Diese Mappings ermöglichen später die Rückverfolgung von View-Objekten auf das zugehörige Datenmodell. Sie sind insbesondere für die Selektion, Färbung und Nachbarschaftsanalyse essentiell. Die gesamte Pipeline ist modular aufgebaut und in den Methoden `createMeshViews` (JavaFX) bzw. `createGeometries` (jME) gekapselt.

### 3.3 Topologischer Test mittels Euler-Poincaré-Formel

Zur Überprüfung der topologischen Konsistenz eines Polyeders wird eine erweiterte Form der Euler-Poincaré-Formel herangezogen. Diese berücksichtigt neben Ecken ( $V$ ), Kanten ( $E$ ) und Flächen ( $F$ ) auch komplexere Strukturelemente wie innere Loops ( $L$ ), die Anzahl zusammenhängender Komponenten (Shells  $S$ ) und den Genus ( $G$ ) eines Körpers.

**1. Klassische Euler-Charakteristik** Zunächst wird auf Basis folgender Formel ein erweiterter Klassik-Wert berechnet:

$$\chi_{\text{classic}} = V - E + F - (L - F)$$

Diese Erweiterung der klassischen Formel berücksichtigt, dass jede Fläche mit Löchern zusätzliche Ränder (Loops) erzeugt. Bei einem einfach zusammenhängenden Körper ohne Löcher reduziert sich dieser Ausdruck auf die klassische Euler-Charakteristik.

**2. Korrektur für komplexe Topologien** Da Polyeder auch aus mehreren zusammenhängenden Komponenten (Shells  $S$ ) bestehen oder torusartige Durchdringungen (Genus  $G$ ) besitzen können, wird zusätzlich ein Korrekturterm berücksichtigt. Die vollständig angepasste Euler-Poincaré-Formel lautet:

$$\chi_{\text{adjusted}} = \chi_{\text{classic}} - 2(S - G)$$

Dieser korrigierte Wert sollte bei einem topologisch geschlossenen, konsistenten Körper den Wert 0 ergeben. Die Berechnung erfolgt in der Methode `calcEulerPoincareCharacteristic(Polyhedron)`. Alle Zählwerte werden zuvor durch dedizierte Hilfsmethoden bestimmt:

- `calcV`: Anzahl der Punkte ( $V$ )
- `calcE`: Anzahl der eindeutigen Kanten ( $E$ )
- `calcF`: Anzahl der Flächen ( $F$ )
- `calcL`: Gesamtanzahl der Loops (Außenränder + Löcher)
- `calcS`: Anzahl der Shells – aktuell immer 1

Der Genus  $G$  wird anschließend aus  $\chi_{\text{classic}}$  und  $S$  berechnet. Das Ergebnis – inklusive der vollständig ausgewerteten Formel – wird im jeweiligen `Polyhedron`-Objekt als String gespeichert.

**3. Prüfung auf Geschlossenheit** Unabhängig von der Zählformel wird über die Methode `isWatertight(Polyhedron)` geprüft, ob jede `HalfEdge` eine gültige `twin`-Referenz besitzt, die korrekt zurückverweist. Nur wenn diese Bedingung erfüllt ist und  $\chi_{\text{adjusted}} = 0$  gilt, wird das Polyeder als „watertight“ klassifiziert.

Diese zweistufige Prüfung kombiniert strukturelle Konsistenz (über Twin-Beziehungen) mit einer topologischen Validierung der Flächenstruktur.

## 4 Unit Tests

Zur Sicherstellung der Korrektheit der zentralen Algorithmen wurden gezielte Unit-Tests mit JUnit 5 implementiert. Der Fokus lag dabei auf der topologischen Analyse (BFS und Euler-Poincaré-Test) sowie auf die Prüfung der Vollständigkeit und Konsistenz der Half-Edge-Verknüpfungen (Watertightness)“

### 4.1 Teststrategie

Anstelle einer flächendeckenden Testabdeckung wurde eine gezielte Teststrategie gewählt, die sich auf zentrale Operationen der Systemlogik konzentriert:

- Topologische Analyse per Breitensuche (BFS)
- Validierung der Euler-Poincaré-Charakteristik
- Prüfung auf geschlossene Kantenzuordnung (Watertightness)

Dabei wurden sowohl positive Tests (gültige Topologien) als auch Negativtests (fehlerhafte Modelle) berücksichtigt.

### 4.2 Testfälle

**EulerOperationsTest** Diese Testklasse prüft die Korrektheit der berechneten Euler-Werte und die Topologie-Klassifikation.

- `testIcosahedronIsWatertight()`: prüft, ob ein reguläres, geschlossenes 20-Flächen-Polyeder korrekt als watertight erkannt wird.
- `testTorusGenusOne()`: verifiziert, dass ein torusförmiges Modell korrekt mit Genus 1 analysiert wird.
- `testNonWatertightCubeFails()`: prüft, ob ein absichtlich fehlerhaftes Modell (mit fehlenden Twin-Referenzen) korrekt als *nicht watertight* erkannt wird.

**BFSAnalysisTest** Dieser Testfall überprüft die Nachbarschaftsanalyse auf einem komplexeren Polyeder mit Genus 2.

- `testBFSLevelCountCuboidGenus2()`: analysiert ein Polyeder mit zwei Tunnelstrukturen. Es wird überprüft, ob alle Flächen per BFS erreicht werden, das Startface korrekt als Level 0 markiert ist und die Nachbarschaft mindestens drei unterschiedliche Level aufweist.